

Module 12 - chapter 23

Complete CI/CD with Terraform - Part 2

The installation documentation for Terraform can be found here: <https://developer.hashicorp.com/terraform/downloads>

The project files used in this lecture can be found here:

- Terraform Learn: <https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/eks>
- Java-maven-app: <https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/java-maven-app>

The installation documentation for Docker Compose standalone can be found here: <https://docs.docker.com/compose/install/standalone/>

Create a new multi-branch pipeline

I used my-multi-branch-pipeline as reference to create terraform-pipeline and also I review Module 8 - chapter 15

Webhooks - Trigger Pipeline Jobs automatically to add the webhook credential to the new pipeline. my-multi-branch-pipeline has a credential called my-pipeline which is use

After I created the new multi-branch pipeline Jenkins registered the webhook in the repo automatically (my terraform repo didn't have a webhook before this)

— tangent —

Your Jenkins server is configured to automatically register a webhook on new GitHub repos when it detects them or when a new pipeline/job is created. This is standard Jenkins + GitHub integration behavior — Jenkins calls the GitHub API to create the webhook itself so it gets notified of pushes, PRs, etc.

So the flow is: you create a new repo → Jenkins notices (or you create a job pointing at it) → Jenkins automatically calls the GitHub API to register its webhook on that repo.

You don't need to create webhooks manually — Jenkins is handling it for you. The GitHub CLI OAuth app is unrelated, that's just the gh command line tool.

Q:

did this happen when I created the multi branch pipeline? what I mean is that probably the repo didn't have the webhook and then I created the multibranch pipeline where I input the repo address and the jenkins register the webhook in the repo?

A:

Yes, exactly. That's the sequence:

1. New repo created — no webhook
2. You created the Multibranch Pipeline in Jenkins and entered the repo URL
3. Jenkins called the GitHub API to register its webhook on that repo automatically

The Multibranch Pipeline needs to know about new branches and PRs in real time, so it registers the webhook as part of the setup so GitHub can notify it of events. You don't have to do it manually because Jenkins does it for you at job creation time.

Q:

where is the configuration of my jenkins that tells to do the registry automatically?

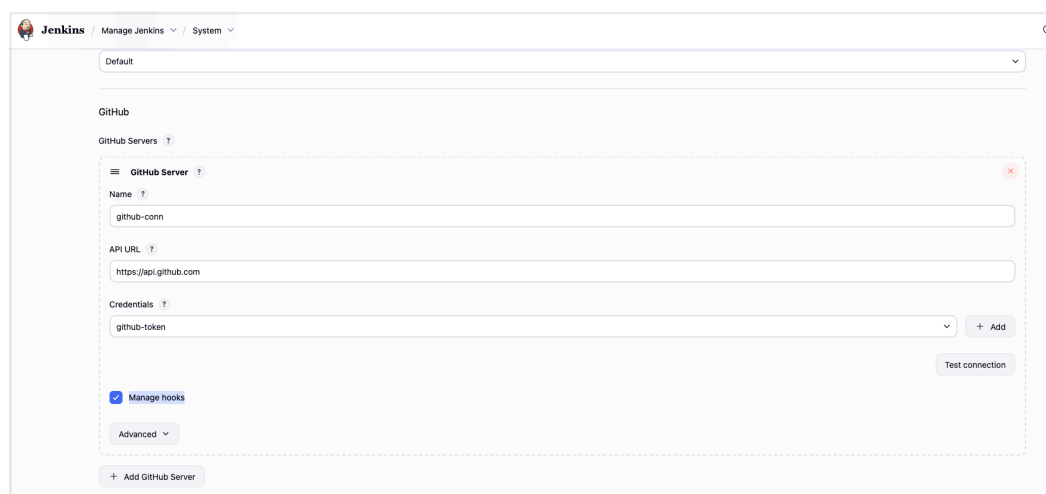
A:

In your Jenkins UI, go to:

Manage Jenkins → System and look for the GitHub section. There you'll find a GitHub Server configuration with credentials (a Personal Access Token). When this is configured, Jenkins has the permission and knows to auto-register webhooks.

The specific setting is usually called "Manage hooks" — it's a checkbox next to the GitHub server entry. If that's checked, Jenkins will automatically create/manage webhooks on repos whenever a job is pointed at them.

That's the setting that enables the behavior you're seeing.



— end tangent —

Create the ssh key pair

In module 12 - chapter 13 we created a key pair via terraform in our laptops

```
resource "aws_key_pair" "ssh-key" {
  key_name   = "server-key"
  public_key = file("~/ssh/id_rsa.pub")
}
```

And also we can do it using AWS console.

For this demo we will use AWS to create the key pair (instead of using tf) and then we will pass it to jenkins container.

EC2 > Key pairs > Create key pair

Create key pair [Info](#)

Key pair
A key pair, consisting of a private key and a public key, is a set of security credentials that you use to prove your identity when you connect to an Amazon EC2 instance.

Name
myapp-key-pair
The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type [Info](#)
☒ RSA ☐ ED25519

Private key file format
☒ .pem
For use with OpenSSH
☐ .ppk
For use with PuTTY

Tags - optional
No tags associated with the resource.
[Add new tag](#)
You can add up to 50 more tags.

And the key-pair pem file has been downloaded automatically upon creation in AWS

Name	Type	Created	Fingerprint
myapp-key-pair	rsa	2026/06/26 08:47 GMT+1	0f:87:f4:5d:13:b9:63:8a:e7:f7:79:55:35:15:32:54:4e...
jenkins-server	rsa	2026/06/26 14:48 GMT+1	2e:67:11:40:9f:bb:51:a8:71:3e:60:2e:6a:3d:1e:63:e5...

Now we create a new SSH credential in Jenkins using the contents of the pem file:

Jenkins / terraform-pipeline / Credentials / Folder / Global credentials (unrestrict...

New credentials

Kind
SSH Username with private key

ID ?
server-ssh-key

Description ?

Username
ec2-user

☐ Treat username as secret ?

Private Key
☒ Enter directly

Key
v3dLvReE6v0Zwth010XjB8jwRwM0xGjMCuotUi+1UR/Z62TVuST8jIY4/LYD1B
F2G/ruTd/ltBC1ppeSWF7/MYswpsz8ZMFQeQc9pRAoGBAKbHlyGfQlyIoFaJYNG8
nrSMebkNQIjkdUyVYfNF8zUmV25mNjKKD0EYziTuWcbD0A/HnXX9LyXXhRYrDY
U1MuLZrNS0wmfApB7agEKDSaVFRFR9tLVZiN0Uix8UrgKHyCuyk+tgq+aEcMecLc
jBaHLZmTCjvRrfylnJTP0JBtAoGAZ4+MKJH6150BKvJHeZUliq7UcyX0s/ueWy7B
LCXkKkWAJwo2y3+041RZE+0VmkCKhca1zyA3Aj31dhjFD00vww+LFE/hpCG/cwf
w02N8YbMVhb1LEwMDIAPwRL5uS+*1L1agqp5s2kzwq/IO1TWCDM/zZ5K+rG0y3Z
/gWkuREcGYAxOuGxqamS0W4iZTX2ER1UTVCEa+BHJ53bpKL7a40AAWq8rc/pztW
h/LwuM7CbbVh0ToxZjLLFDAdaamaKpn134BSJF0qy29rmmIZfjvLDP8W6LF0JxzE
erRLYZs0LTZSYIX75mhEpv00NuWoeby9NR4kgmxLlvY08gr7aU4bfw==
-----END RSA PRIVATE KEY-----

Passphrase

Create

First step is done

Steps to do

- 1) create SSH key-pair 
- 2) install TF inside Jenkins container
- 3) TF configuration to provision server
- 4) adjust Jenkinsfile



Now Jenkins will be able to SSH into the EC2 server to deploy the application using this private key.

Install terraform inside Jenkins container

I ssh into the droplet

-> ssh root@165.22.116.21

UTC 2023

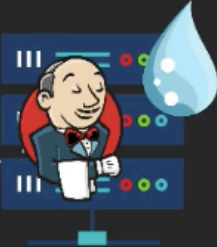
id in: 0

IPs for docker0: 172.17.0.1

IPs for eth0: 165.232.114

IPs for eth0: 10.19.0.5

IPs for eth1: 10.114.0.2



Remember: Jenkins runs as Docker container on our remote DigitalOcean droplet

week old.

-> docker ps

```
root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
56efcedae10e   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 2 weeks ago   Up 2 weeks   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
naughty_dijkstra
```

We can see the container running jenkins and now we enter jenkins container as user root:

-> docker exec -it -u 0 56efcedae10e bash

-u 0 -> user zero = user root -> super user

```
root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
56efcedae10e   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 2 weeks ago   Up 2 weeks   0
tcp naughty_dijkstra
root@jenkins-server:~# docker exec -it -u 0 56efcedae10e bash
root@56efcedae10e:/#
```

We check the OS of the container

-> cat etc/os-release

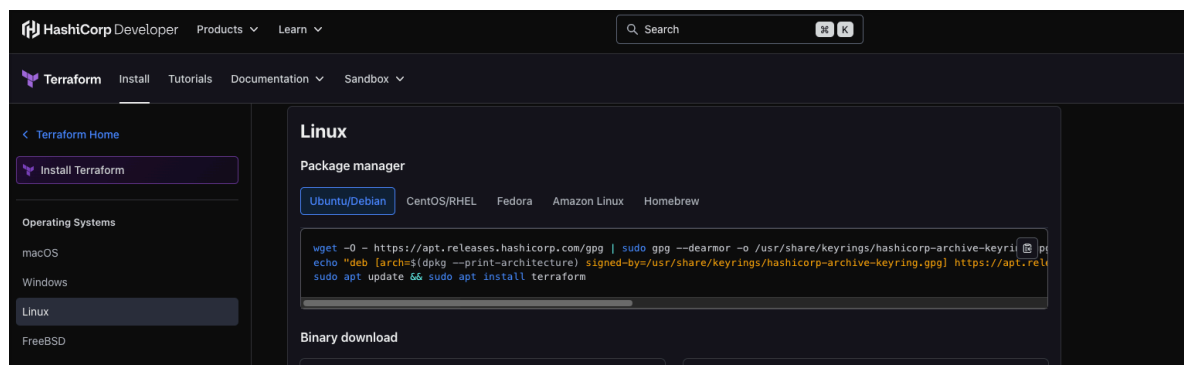
```

root@56efcedae10e:/# cat etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.1
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
root@56efcedae10e:/#

```

And we have a Debian distribution -> linux

Then we follow the installation instructions based on the OS we have -> <https://developer.hashicorp.com/terraform/install>



First command will add the HashiCorp key and write it to a file on our Jenkins Server

-> `wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg`

Error:

```

root@56efcedae10e:/# wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
bash: wget: command not found
bash: sudo: command not found

```

Problem:

You're running as root inside the container already (that's why sudo isn't found), and wget isn't installed. Try this instead:

-> `apt-get update && apt-get install -y wget gpg`

Then re-run the original command but without sudo.

```
-> wget -O - https://apt.releases.hashicorp.com/gpg | gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

```
root@56efcedae10e:/# wget -O - https://apt.releases.hashicorp.com/gpg | gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
File '/usr/share/keyrings/hashicorp-archive-keyring.gpg' exists. Overwrite? (y/N) --2026-06-26 09:44:59-- https://apt.releases.hashicorp.com/gpg
Resolving apt.releases.hashicorp.com (apt.releases.hashicorp.com)... 18.245.143.73, 18.245.143.93, 18.245.143.105, ...
Connecting to apt.releases.hashicorp.com (apt.releases.hashicorp.com)|18.245.143.73|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3980 (3.9K) [binary/octet-stream]
Saving to: 'STDOUT'

-                  100%[=====] 3.89K --.-KB/s in 0s

2026-06-26 09:44:59 (236 MB/s) - written to stdout [3980/3980]

Overwrite? (y/N) y
root@56efcedae10e:/#
```

Next command adds the official hashicorp repository to the list of installation sources

installation sources (also called package sources or apt sources) — the list of places where apt looks to download software.

```
-> iecho "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $
(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs)
main" | tee /etc/apt/sources.list.d/hashicorp.list
```

AI says:

The command you have adds HashiCorp's repository as a new source. The main sources on a Debian/Ubuntu system are:

- /etc/apt/sources.list — the main file, usually contains the OS's official repos (Ubuntu, Debian mirrors)
- /etc/apt/sources.list.d/ — a directory for additional sources, one file per third-party repo. This is where the command is writing hashicorp.list

So when you run apt-get install terraform, apt knows to look in HashiCorp's repo because of that file.

Common examples of third-party sources added this way:

- HashiCorp (Terraform, Vault, Consul)
- Docker
- Google (kubect!, Chrome)
- NodeSource (Node.js)
- Jenkins itself

The GPG key you added earlier is the trust mechanism — apt uses it to verify that packages from that source actually came from HashiCorp and weren't tampered with.

I am using Debian trixie (see -> cat etc/os-release) not ubuntu so I need to edit the command:

```
-> echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $
(grep -oP '(?<=VERSION_CODENAME=).*' /etc/os-release) main" | tee /etc/apt/
sources.list.d/hashicorp.list
```

```
root@56efcedae10e:/# echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=
VERSION_CODENAME=).*' /etc/os-release) main" | tee /etc/apt/sources.list.d/hashicorp.list
deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com trixie main
```

Next we install terraform

-> apt update

```
root@56efcedae10e:/# apt-get update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Get:4 https://apt.releases.hashicorp.com trixie InRelease [12.9 kB]
Get:5 https://apt.releases.hashicorp.com trixie/main amd64 Packages [243 kB]
Fetched 256 kB in 0s (994 kB/s)
Reading package lists... Done
```

-> apt install terraform

```
root@56efcedae10e:/# apt install terraform
Installing:
  terraform

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 33
  Download size: 34.8 MB
  Space needed: 117 MB / 69.8 GB available

Get:1 https://apt.releases.hashicorp.com trixie/main amd64 terraform amd64 1.15.7-1 [34.8 MB]
Fetched 34.8 MB in 0s (96.5 MB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79, <STDIN> line 1.)
debconf: falling back to frontend: Readline
Selecting previously unselected package terraform.
(Reading database ... 10052 files and directories currently installed.)
Preparing to unpack .../terraform_1.15.7-1_amd64.deb ...
Unpacking terraform (1.15.7-1) ...
Setting up terraform (1.15.7-1) ...
```

-> terraform -v

```
root@56efcedae10e:/# terraform -v
Terraform v1.15.7
on linux_amd64
root@56efcedae10e:/#
```

— tangent —

An alternative to install terraform and skip **apt** entirely is to just download the binary directly:

apt-get update && apt-get install -y curl unzip

curl -fsSL https://releases.hashicorp.com/terraform/1.11.4/

```
terraform_1.11.4_linux_amd64.zip -o terraform.zip && \  
unzip terraform.zip && \  
mv terraform /usr/local/bin/ && \  
rm terraform.zip
```

Then verify:
terraform --version

This works regardless of the OS/distro and is actually the approach most people use in containers anyway.

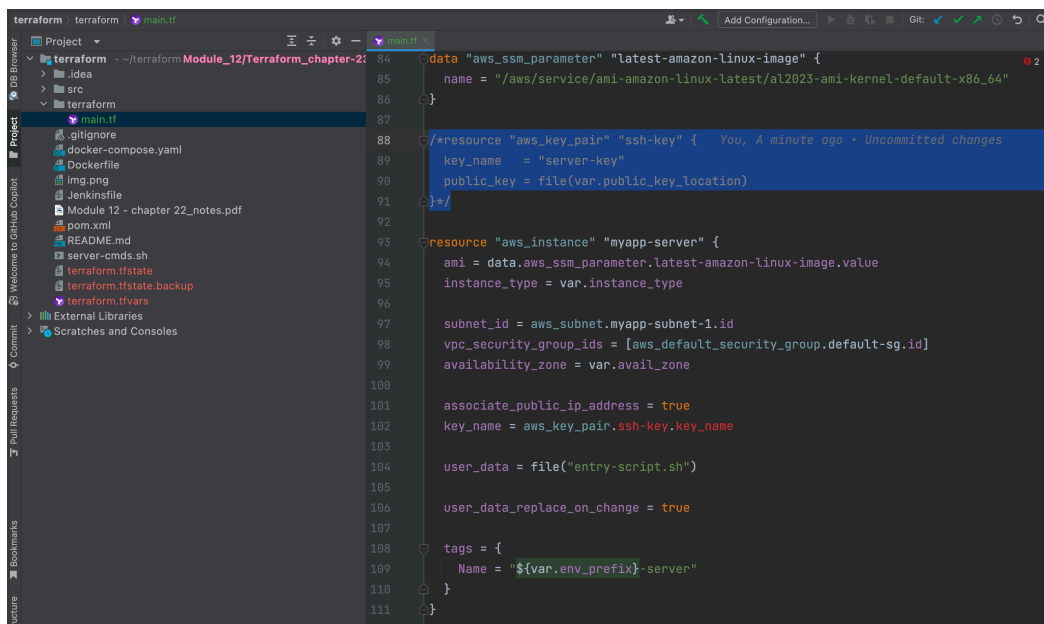
— end tangent —

Terraform configuration files

We will add the tf config files in our application code

I create a new folder called 'terraform' and inside I create a file called 'main.tf' and I copy the code from Module 12 - chapter 14 'main.tf' and paste it in the new file

And we remove the key pair (line 88) resource from the code (we already did this part manually using AWS)



```
84 data "aws_ssm_parameter" "latest-amazon-linux-image" {  
85   name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"  
86 }  
87  
88 /*resource "aws_key_pair" "ssh-key" {  
89   key_name = "server-key"  
90   public_key = file(var.public_key_location)  
91 }*/  
92  
93 resource "aws_instance" "myapp-server" {  
94   ami = data.aws_ssm_parameter.latest-amazon-linux-image.value  
95   instance_type = var.instance_type  
96  
97   subnet_id = aws_subnet.myapp-subnet-1.id  
98   vpc_security_group_ids = [aws_default_security_group.default-sg.id]  
99   availability_zone = var.avail_zone  
100  
101   associate_public_ip_address = true  
102   key_name = aws_key_pair.ssh-key.key_name  
103  
104   user_data = file("entry-script.sh")  
105  
106   user_data_replace_on_change = true  
107  
108   tags = {  
109     Name = "${var.env_prefix}-server"  
110   }  
111 }
```

And we update line 97 📌

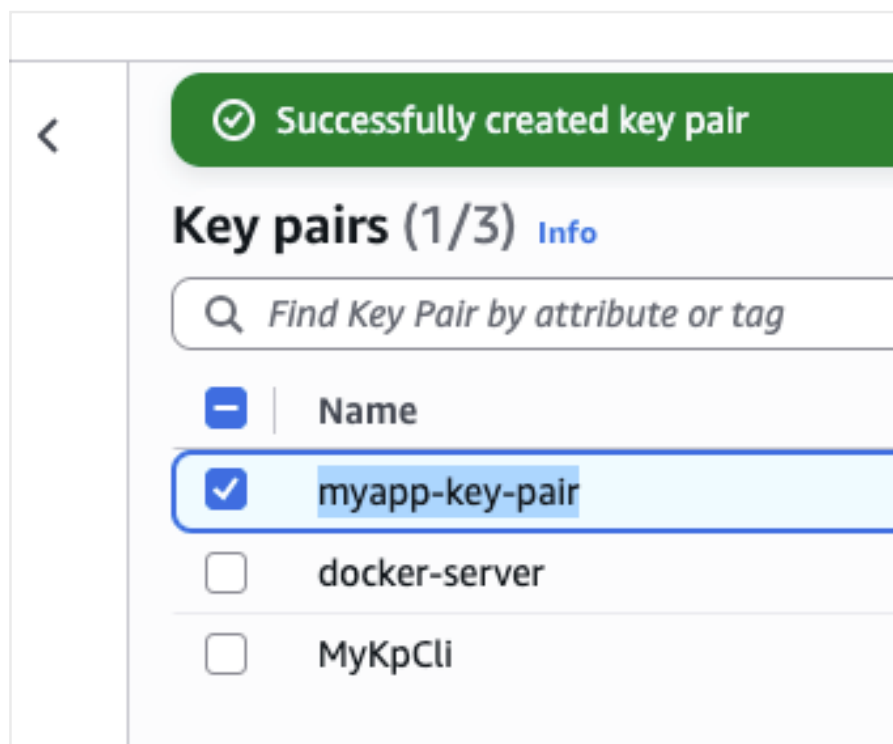
```

87
88 resource "aws_instance" "myapp-server" {
89     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
90     instance_type = var.instance_type
91
92     subnet_id = aws_subnet.myapp-subnet-1.id
93     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
94     availability_zone = var.avail_zone
95
96     associate_public_ip_address = true
97     key_name = aws_key_pair.ssh-key.key_name
98
99     user_data = file("entry-script.sh")
100
101     user_data_replace_on_change = true
102
103     tags = {
104         Name = "${var.env_prefix}-server"
105     }
106 }

```

You, A minute ago • Uncommitted changes

to use the AWS key that we created earlier -> myapp-key-pair



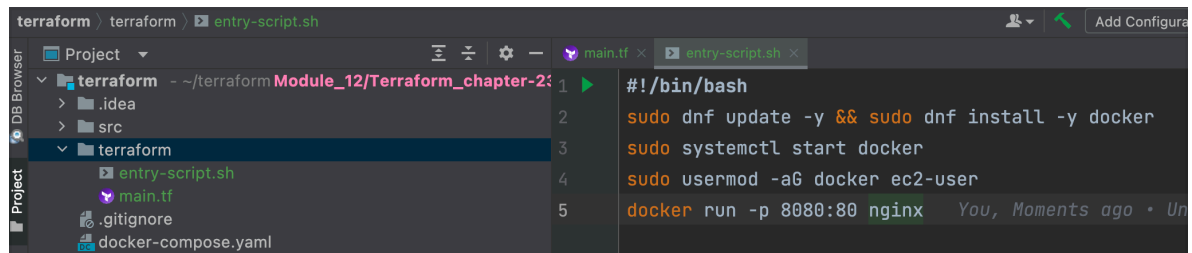
And now our code looks like this

```

95
96     associate_public_ip_address = true
97     key_name = "myapp-key-pair"
98

```

In line 99 we call 'entry-script.sh'; so we need that file in our project. I copy it and paste it



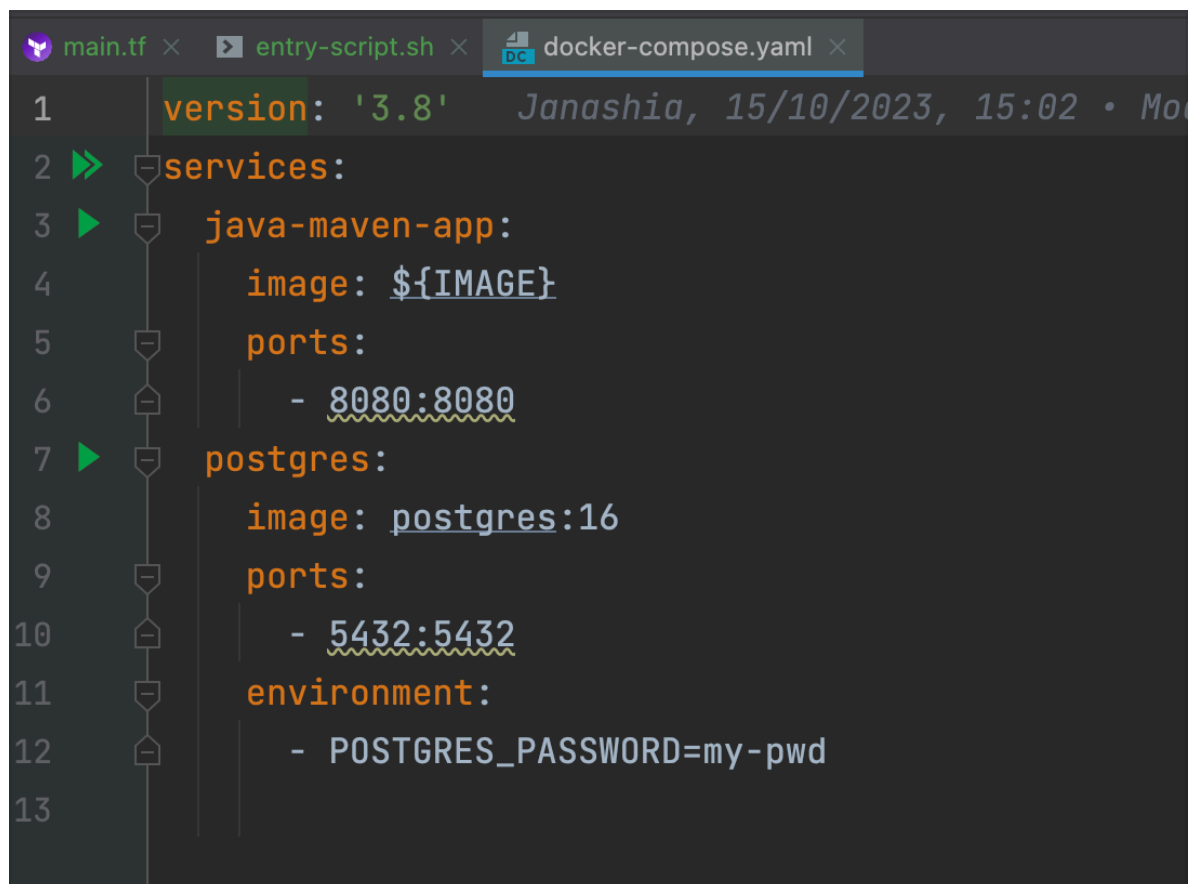
The screenshot shows an IDE window with the file 'entry-script.sh' open. The left sidebar shows a project structure with 'terraform' and 'src' folders. The main editor displays the following script content:

```

1  #!/bin/bash
2  sudo dnf update -y && sudo dnf install -y docker
3  sudo systemctl start docker
4  sudo usermod -aG docker ec2-user
5  docker run -p 8080:80 nginx

```

But we want to deploy and execute docker compose which will run two containers one for our app and another for the Postgres DB instead of getting docker to run nginx container.



The screenshot shows an IDE window with the file 'docker-compose.yml' open. The left sidebar shows the project structure. The main editor displays the following docker-compose configuration:

```

1  version: '3.8'
2  services:
3    java-maven-app:
4      image: ${IMAGE}
5      ports:
6        - 8080:8080
7    postgres:
8      image: postgres:16
9      ports:
10       - 5432:5432
11      environment:
12        - POSTGRES_PASSWORD=my-pwd
13

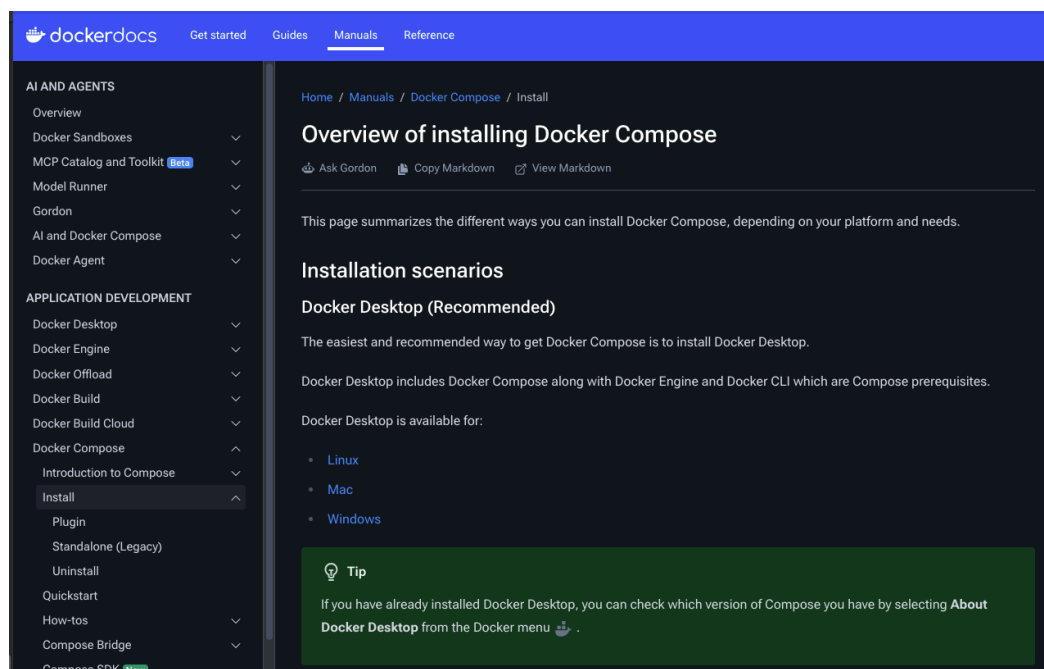
```

We will copy the docker-compose.yml file into the EC2 instance and get docker-compose to run all the containers we need via sshagent (see lines 49-52)

```
Jenkinsfile M x main.tf U entry-script.sh U
30     echo 'building docker image...'
31     buildImage(env.IMAGE_NAME)
32     dockerLogin()
33     dockerPush(env.IMAGE_NAME)
34 }
35 }
36 }
37 stage("provision server") {
38     // tf provision server
39     sh "terraform init"
40 }
41 stage("deploy") {
42     steps {
43         script {
44             echo 'deploying docker image to EC2...'
45
46             def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
47             def ec2Instance = "ec2-user@35.180.251.121"
48
49             sshagent(['ec2-server-key']) {
50                 sh "scp server-cmds.sh ${ec2Instance}:/home/ec2-user"
51                 sh "scp docker-compose.yaml ${ec2Instance}:/home/ec2-user"
52                 sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
53             }
54         }
55     }
56 }
```

So we need to install docker compose as well.

<https://docs.docker.com/compose/install>



— tangent —

In Module 11 - chapter 9 I created the jenkins container using:


```
-> docker run -p 8080:8080 \
  -p 50000:50000 -d \
  -v /root/.kube:/var/jenkins_home/.kube \
  -v /usr/local/bin/kubectrl:/usr/local/bin/kubectrl \
  -v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /usr/bin/docker:/usr/bin/docker \
  jenkins/jenkins:its
```

Q:

I need to write the code to install docker compose in my docker-compose.yaml the teacher is using docker-compose stand alone but the current documentation says that the standalone is legacy.

A:

You have two options:

Option 1: Same pattern, mount the binary (quickest)

Download docker-compose on the host and mount it in, just like you did with docker:

on the host

```
curl -SL https://github.com/docker/compose/releases/download/v2.36.2/
docker-compose-linux-x86_64 -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose
```

Then add to your docker run:

```
-v /usr/local/bin/docker-compose:/usr/local/bin/docker-compose
```

Option 2: Create a Jenkins Dockerfile (cleaner, recommended)

Create a Dockerfile that extends the official image:

```
FROM jenkins/jenkins:its
```

USER root

```
RUN apt-get update && apt-get install -y docker-compose-plugin
```

USER jenkins

Then build and run from that image instead of jenkins/jenkins:its.

Option 2 is better long-term — everything Jenkins needs is self-contained in the image rather than depending on what's installed on the host. But Option 1 is faster if you just need it working now.

Q:

why I need this step: Then add to your docker run:

```
-v /usr/local/bin/docker-compose:/usr/local/bin/docker-compose
```

A:

Because the binary is installed on the host, not inside the container. Without mounting it, the Jenkins container has no idea it exists.

It's the same reason you mounted `/usr/bin/docker` — Docker was installed on the host and you needed to make it accessible inside the container by mounting it in.

Q:

so if for the demo I don't mount it it will work but if the container dies I lose everything

A:

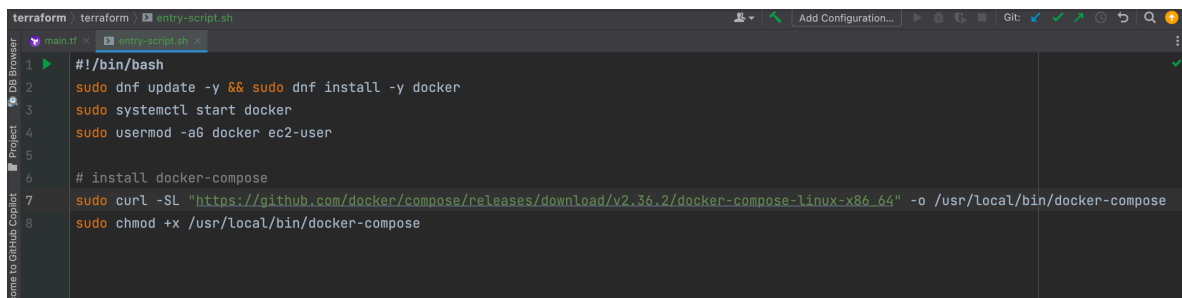
Exactly. If you don't mount it:

- It works for the demo (you installed it manually inside the running container)
- Container dies → everything gone, you have to SSH in and install it again

That's precisely why the Dockerfile approach is better — the install is baked into the image, so every time the container starts it's already there.

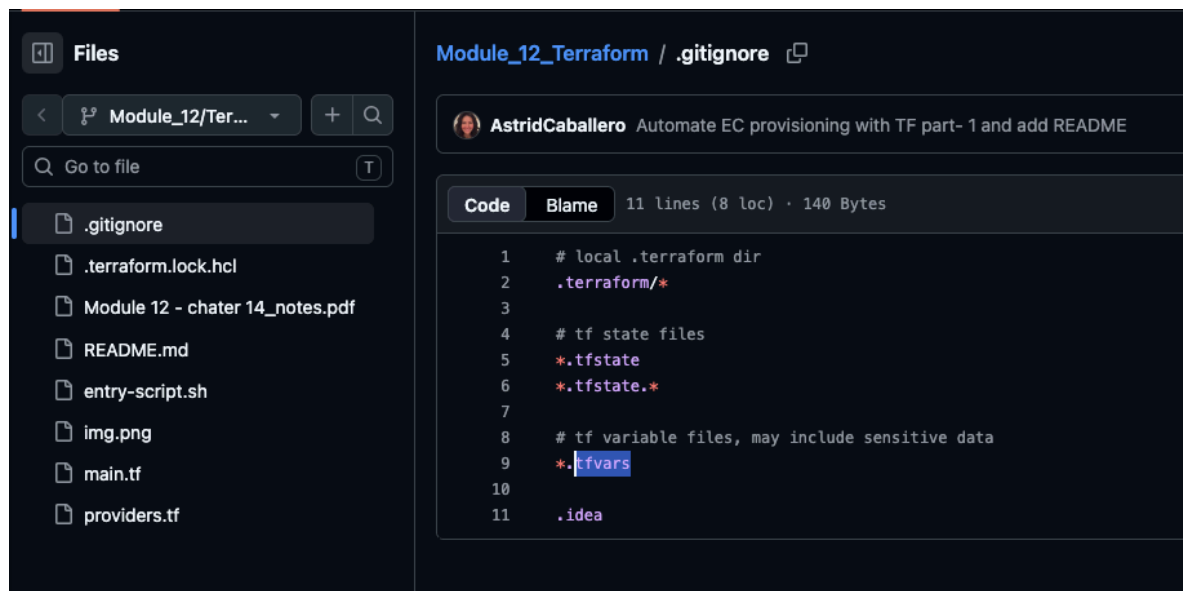
— end tangent —

So for now I will use part of option 1 from the tangent above but I won't mount docker-compose in the container for this demo



```
terraform terraform entry-script.sh
main.tf x entry-script.sh x
1 #!/bin/bash
2 sudo dnf update -y && sudo dnf install -y docker
3 sudo systemctl start docker
4 sudo usermod -aG docker ec2-user
5
6 # install docker-compose
7 sudo curl -SL "https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64" -o /usr/local/bin/docker-compose
8 sudo chmod +x /usr/local/bin/docker-compose
```

In our previous demos we placed the values of the variables in a file called `terraform.tfvars` this `.tfvars` file is in `.gitignore` hence it was not pushed to the repo as it contained sensitive information



So how do we provide the values to jenkins container environment? How do we set them?

Let's take a look and the variables and remove the public key one because we didn't use it in this demo:



So now we have some variables whose values don't change much so we can give them default values. Remember that default values can be overridden later.

```

5  -variable vpc_cidr_block {
6      default = "10.0.0.0/16"
7  }
8  -variable subnet_cidr_block {
9      default = "10.0.10.0/24"
10 }
11 -variable avail_zone {
12     default = "eu-west-2a"
13 }
14 -variable env_prefix {
15     default = "dev" | You, Moments ago
16 }
17     variable my_ip {}
18     variable instance_type {}

```

Regarding my ip address, I still want to be able as the human to ssh into my EC@ instance, so I will pass my IP address as the default

```

16     }
17     -variable my_ip {
18         default = "140.228.47.252"
19     }

```

And for the instance type

```

19 }
20 variable instance_type {
21     default = "t2.micro"
22 }
23

```

So now with the defaults we don't have to provide the values all the time, only we will provide values that we want to override

We can also parameterise the region



The screenshot shows a code editor with two tabs: 'main.tf' and 'entry-script.sh'. The 'main.tf' tab is active, displaying a Terraform configuration for the AWS provider. The code is as follows:

```

1 provider "aws" {
2     region = "eu-west-2"
3 }

```

The 'region' value 'eu-west-2' is highlighted in blue. A lightbulb icon is visible next to the 'provider' keyword, indicating an IDE suggestion or linting feature.

So now I have:

```
main.tf x entry-script.sh x
1 provider "aws" {
2     region = var.region
3 }
4
5 variable vpc_cidr_block {default = "10.0.0.0/16"}
8 variable subnet_cidr_block {default = "10.0.10.0/24"}
11 variable avail_zone {
12     default = "eu-west-2a"
13 }
14 variable env_prefix {
15     default = "dev"
16 }
17 variable my_ip {
18     default = "140.228.47.252"
19 }
20 variable instance_type {
21     default = "t2.micro"
22 }
23 variable region { You, Moments ago • Uncommitted changes
24     default = "eu-west-2"
25 }
```

An now I will move all those variables to a new file called 'variables.tf'

```
terraform terraform main.tf variables.tf
1 provider "aws" {
2     region = var.region
3 }
4
5 resource "aws_vpc" "myapp-vpc" {
6     cidr_block = var.vpc_cidr_block
7     tags = {
8         Name: "${var.env_prefix}-vpc"
9     }
10 }
11
12 resource "aws_subnet" "myapp-subnet-1" {
13     vpc_id = aws_vpc.myapp-vpc.id
14     cidr_block = var.subnet_cidr_block
15     availability_zone = var.avail_zone
16     tags = {
17         Name: "${var.env_prefix}-subnet-1"
18     }
19 }
20
21 variable vpc_cidr_block {default = "10.0.0.0/16"}
22 variable subnet_cidr_block {default = "10.0.10.0/24"}
23 variable avail_zone {default = "eu-west-2a"}
24 variable env_prefix {default = "dev"}
25 variable my_ip {default = "140.228.47.252"}
26 variable instance_type {default = "t2.micro"}
27 variable region {default = "eu-west-2"} You, Moments ago • Unc
```

Now the main.tf is cleaner.

We will keep outputs in the main.tf as we only have one.

```
71 output "ec2_public_ip" {  
72     value = aws_instance.myapp-server.public_ip  
73 }  
74
```

You, Moments ago • Uncommitted changes

Steps to do



- 1) create SSH key-pair
- 2) install TF inside Jenkins container
- 3) TF configuration to provision server
- 4) adjust Jenkinsfile



Provision stage in Jenkinsfile

The final part is to adjust jenkinsfile for the CI/CD pipeline.

So let's add the stage "provision server" (line 37)

```

erraform > Jenkinsfile
main.tf x Jenkinsfile x
23     buildJar()
24     }
25     }
26     }
27     stage("build image") {
28     steps {
29     script {
30     echo 'building docker image...'
31     buildImage(env.IMAGE_NAME)
32     dockerLogin()
33     dockerPush(env.IMAGE_NAME)
34     }
35     }
36     }
37     stage("provision server") {
38     steps {
39     script {
40     | You, Moments ago • Uncommitted changes
41     }
42     }
43     }
44     stage("deploy") {
45     steps {
46     script {
47     echo 'deploying docker image to EC2...'
48
49     def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
50     def ec2Instance = "ec2-user@$35.180.151.121"

```

And inside the script we will execute Terraform commands.

And we need to execute the commands from inside the directory where the terraform configuration is which is the terraform folder and for that we use 'dir'

```

36     }
37     stage("provision server") {
38     steps {
39     script {
40     | You, A minute ago •
41     dir('terraform') {
42     }
43     }
44     }
45     }

```

Now that we are inside the folder 'terraform' we can execute the terraform commands

-> terraform init


```

37     stage("provision server") {
38         steps {
39             script {
40                 dir('terraform') {
41                     sh "terraform init"
42                 }
43             }
44         }
45     }

```

-> terraform apply --auto-approve

```

36     }
37     stage("provision server") {
38         steps {
39             script {
40                 dir('terraform') {
41                     sh "terraform init"
42                     sh "terraform apply -auto-approve"
43                 }
44             }
45         }
46     }

```

But for terraform 'apply' to work we need terraform and Jenkins Server need to authenticate with AWS because we are creating resources in AWS so we need AWS to let us in first.

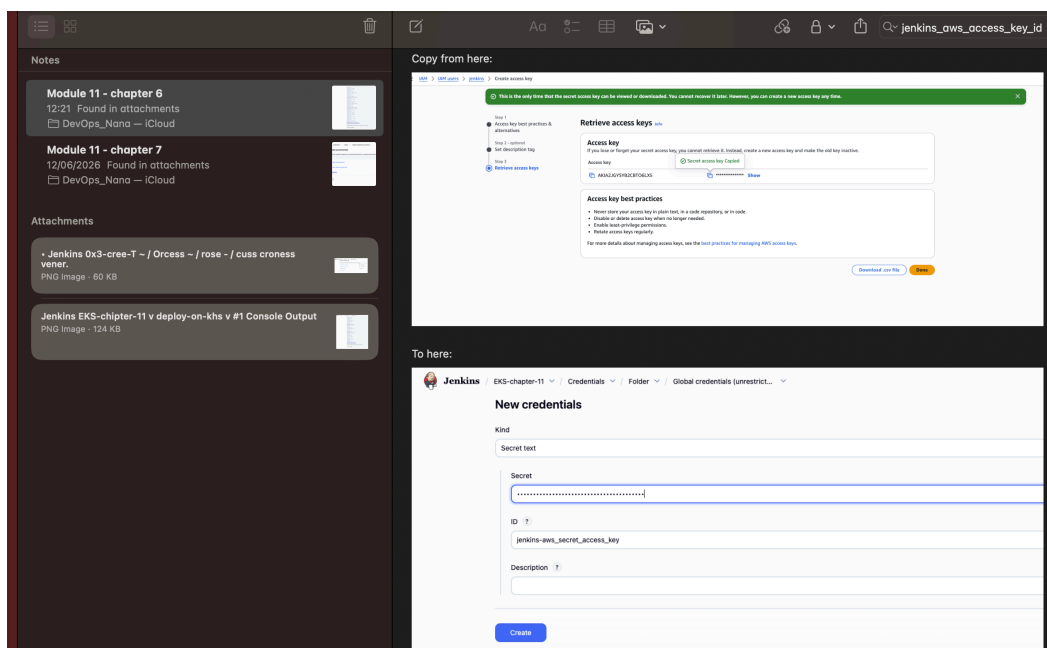
In terraform before we set the credentials inside the 'provider' block. The credentials I used were created in AWS IAM for user 'admin'

```

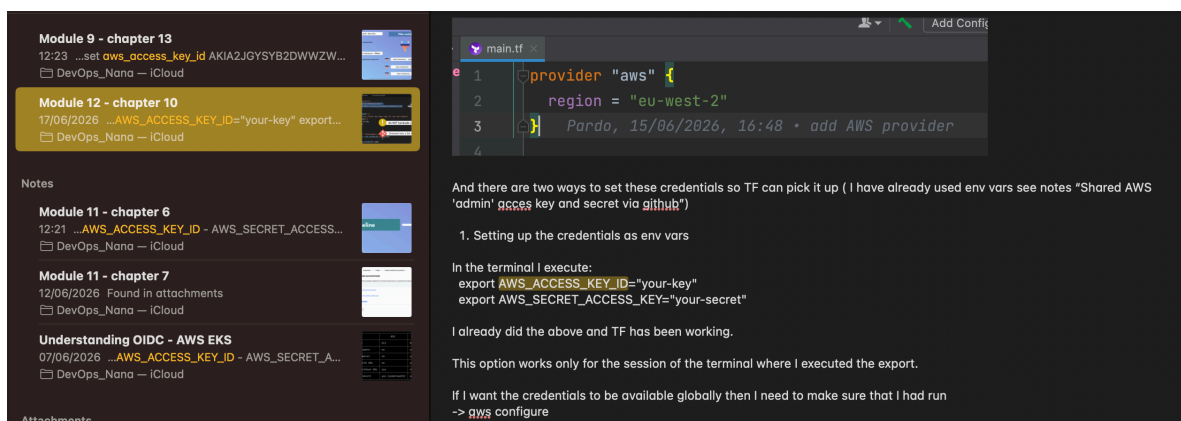
1 provider "aws" {
2     region = "eu-west-2"
3     access_key = "AKIA2JG4..."
4     secret_key = "NicXYQo..."
5 }

```

The credentials we want to use are the ones I created in AWS IAM for user 'jenkins' (I recently change credentials but for user 'admin' because by mistake I push them into GitHub)



And then we set them in our laptop as env variables and also we used aws configure (see Module 12 - chapter 10 Environment Variables in Terraform)



So in jenkinsfile we need to set the env variables so the 'provider' block in

terraform (main.tf) can grab those env vars and connect to AWS.

```
37     stage("provision server") {
38         environment {
39             You, Moments ago • Uncommitted changes
40         }
41         steps {
42             script {
43                 dir('terraform') {
44                     sh "terraform init"
45                     sh "terraform apply -auto-approve"
46                 }
47             }
48         }
49     }
```

And we provide the two variables

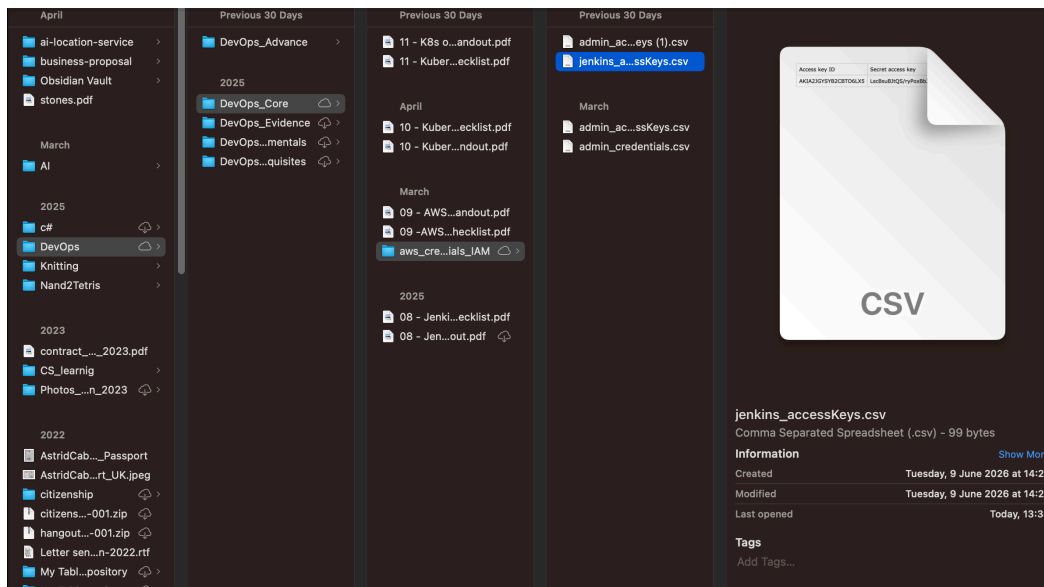
```
37     stage("provision server") {
38         environment {
39             AWS_ACCESS_KEY_ID =
40             AWS_SECRET_ACCESS_KEY = You, A minute a
41         }
```

Nana already have the credentials created in jenkins when we worked in the EKS demos. As I mentioned before I created new credentials for AWS IAM user 'jenkins' so I need to update the current credentials I have

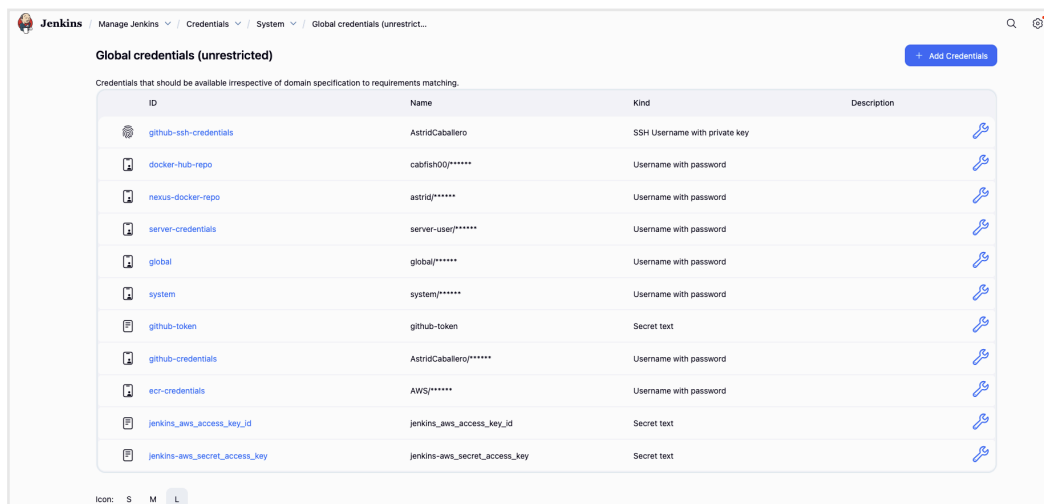
Credentials					
T	P	Store ↓	Domain	ID	Name
		EKS-chapter-11	(global)	jenkins_aws_access_key_id	jenkins_aws_access_key_id
		EKS-chapter-11	(global)	jenkins-aws_secret_access_key	jenkins-aws_secret_access_key
		EKS-chapter-11	(global)	ike-credentials	test-kubeconfig.yaml
		System	(global)	github-ssh-credentials	AstridCaballero
		System	(global)	docker-hub-repo	cabfish00/*****
		System	(global)	nexus-docker-repo	astrid/*****

But as we can see the current credentials I have are part of the EKS-chapter-11 scope, they are not global credentials like Nana's. So I need to create them again but as global

First I find the file with the AWS credentials for 'Jenkins' user



Then I create them in jenkins



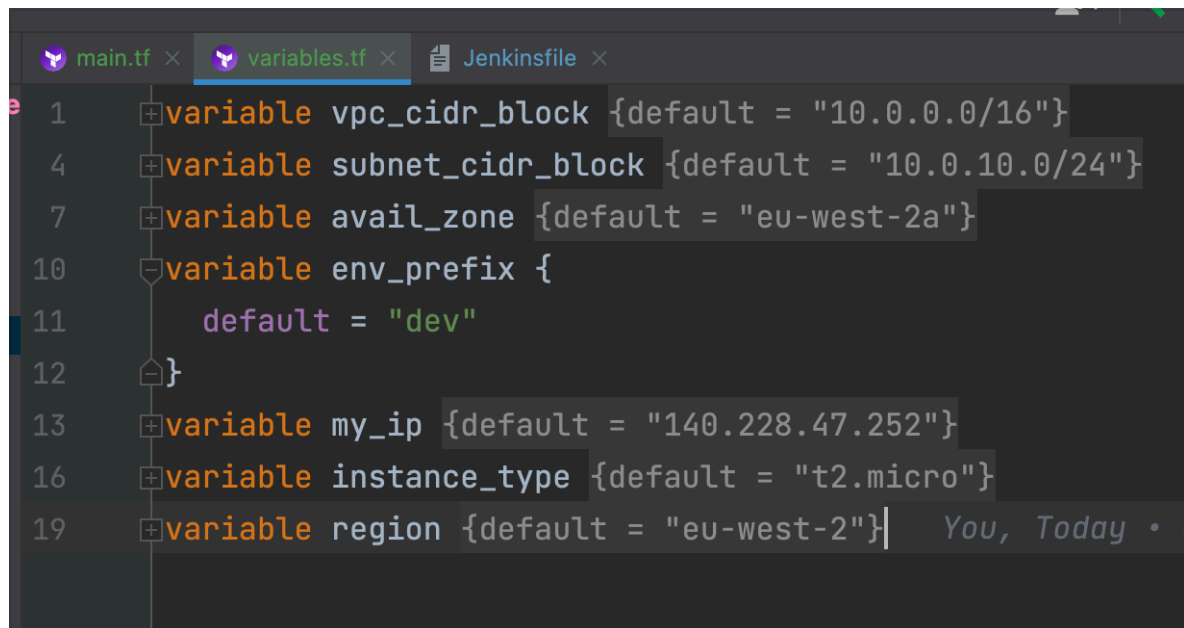
Note: I created these credentials before but locally for EKS-chapter-11 pipeline, I won't delete them (to match the notes rated to that pipeline) although I should do it to avoid confusion as the ids are the same and the secrets are the same so I only need the global credentials.

And now I update the jenkinsfile



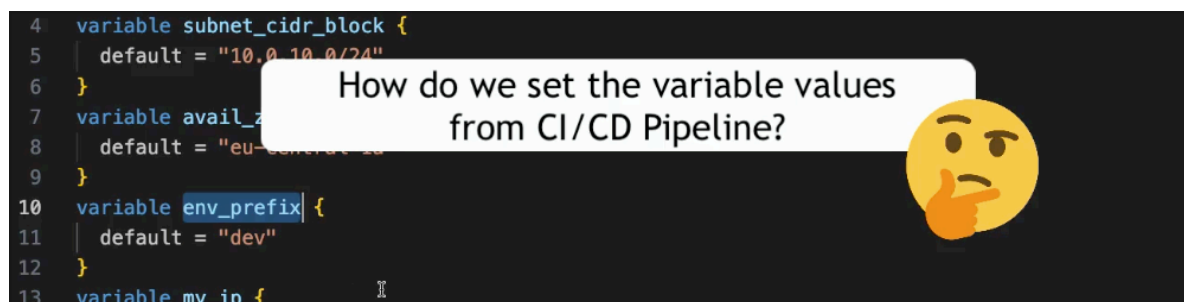
Now jenkins can get authenticated in AWS and execute the terraform commands 🥳

Now we can override variables! Let's say we want to create the infra for a 'test' env instead of 'dev' env (default)



```
1  variable vpc_cidr_block {default = "10.0.0.0/16"}
4  variable subnet_cidr_block {default = "10.0.10.0/24"}
7  variable avail_zone {default = "eu-west-2a"}
10 variable env_prefix {
11     default = "dev"
12 }
13 variable my_ip {default = "140.228.47.252"}
16 variable instance_type {default = "t2.micro"}
19 variable region {default = "eu-west-2"}
```

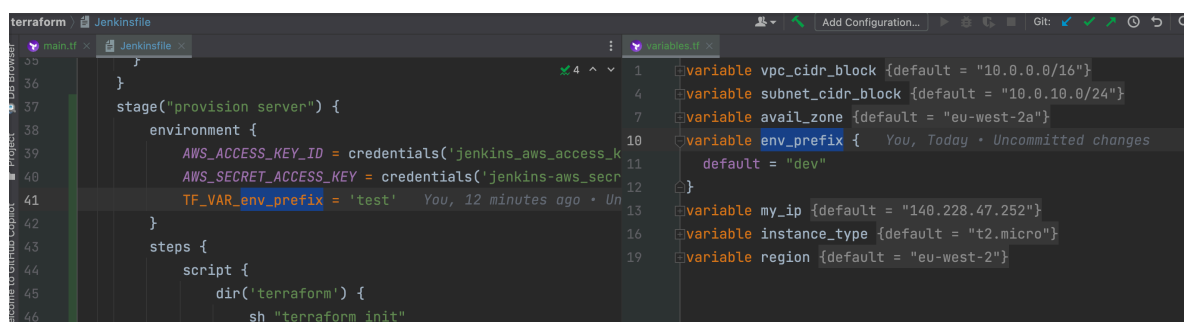
So our CI/CD pipeline is for 'test'



```
4  variable subnet_cidr_block {
5      default = "10.0.10.0/24"
6  }
7  variable avail_zone {
8      default = "eu-west-2a"
9  }
10 variable env_prefix {
11     default = "dev"
12 }
13 variable my_ip {
```

How do we set the variable values from CI/CD Pipeline?

One way is using terraform environment variables -> TF_VAR_<variable_name>



```
36 }
37 stage("provision server") {
38     environment {
39         AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_k
40         AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secr
41         TF_VAR_env_prefix = 'test'
42     }
43     steps {
44         script {
45             dir('terraform') {
46                 sh "terraform init"
```

Deploy Stage in Jenkinsfile

We need to make some adjustments in the 'deploy' stage

First, currently we are hardcoding the ip address of the server

```

52     stage("deploy") {
53         steps {
54             script {
55                 echo 'deploying docker image to EC2...'
56
57                 def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
58                 def ec2Instance = "ec2-user@18.132.18.175"
59
60                 sshagent(['server-ssh-key']) {
61                     sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
62                     sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
63                     sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
64                 }
65             }
66         }

```

But we are provisioning the server with TF so we don't know the IP address that the EC2 instance will have.

So TF will create an EC2 `aws_instance` resource and we need to pass it to jenkinsfile

```

43     steps {
44         script {
45             dir('terraform') {
46                 sh "terraform init"
47                 sh "terraform apply -auto-approve"
48             }
49         }
50     }
51 }
52 stage("deploy") {
53     steps {
54         script {
55             echo 'deploying docker image to EC2...'
56
57             def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
58             def ec2Instance = "ec2-user@18.132.18.175"
59
60             sshagent(['server-ssh-key']) {
61                 sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
62                 sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
63                 sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
64             }
65         }
66     }
67 }
68 }
69 }
70
71 // use SSM parameter store to get the latest Amazon Linux 2 AMI
72 data "aws_ssm_parameter" "latest-amazon-linux-image" {
73     name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel"
74 }
75
76 resource "aws_instance" "myapp-server" {
77     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
78     instance_type = var.instance_type
79
80     subnet_id = aws_subnet.myapp-subnet-1.id
81     vpc_security_group_ids = [aws_default_security_group.default.id]
82     availability_zone = var.avail_zone
83
84     associate_public_ip_address = true
85     key_name = "myapp-key-pair"
86
87     user_data = file("entry-script.sh")
88     user_data_replace_on_change = true
89
90     tags = {
91         Name = "${var.env_prefix}-server"
92     }
93 }

```

How do we pass the resource?

TF has the resource output and we already have an output that returns the `public_ip` of the server

```

43  steps {
44      script {
45          dir('terraform') {
46              sh "terraform init"
47              sh "terraform apply -auto-approve"
48          }
49      }
50  }
51  }
52  stage("deploy") {
53      steps {
54          script {
55              echo 'deploying docker image to EC2...'
56
57              def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
58              def ec2Instance = "ec2-user@18.132.18.175" You, 2 min
59
60              sshagent(['server-ssh-key']) {
61                  sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user/"
62                  sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user/"
63                  sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} $shellCmd"
64              }
65          }
66      }
67  }
68  }
69  }
70
71  resource "aws_instance" "myapp-server" {
72      ami = data.aws_ssm_parameter.latest-amazon-linux-image.v
73      instance_type = var.instance_type
74
75      subnet_id = aws_subnet.myapp-subnet-1.id
76      vpc_security_group_ids = [aws_default_security_group.def
77      availability_zone = var.avail_zone
78
79      associate_public_ip_address = true
80      key_name = "myapp-key-pair"
81
82      user_data = file("entry-script.sh")
83      user_data_replace_on_change = true
84
85      tags = {
86          Name = "${var.env_prefix}-server"
87      }
88  }
89
90  output "ec2_public_ip" {
91      value = aws_instance.myapp-server.public_ip You, Today
92  }

```

So we can get the 'provision server' stage to inform jenkins of the output specifically 'ec2_public_ip'

```

36  }
37  stage("provision server") {
38      environment {
39          AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_k
40          AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secr
41          TF_VAR_env_prefix = 'test'
42      }
43      steps {
44          script {
45              dir('terraform') {
46                  sh "terraform init"
47                  sh "terraform apply -auto-approve"
48                  sh "terraform output ec2_public_ip" You, 20
49              }
50          }
51      }
52  }
53  stage("deploy") {
54      steps {
55          script {
56              dir('terraform') {
57                  sh "terraform init"
58                  sh "terraform apply -auto-approve"
59                  sh "terraform output ec2_public_ip" You, 14 minutes ago • Uncommite
60              }
61          }
62      }
63  }
64  }
65  }
66  }
67  }
68  }
69  }
70
71  resource "aws_instance" "myapp-server" {
72      ami = data.aws_ssm_parameter.latest-amazon-linux-image.v
73      instance_type = var.instance_type
74
75      subnet_id = aws_subnet.myapp-subnet-1.id
76      vpc_security_group_ids = [aws_default_security_group.defa
77      availability_zone = var.avail_zone
78
79      associate_public_ip_address = true
80      key_name = "myapp-key-pair"
81
82      user_data = file("entry-script.sh")
83      user_data_replace_on_change = true
84
85      tags = {
86          Name = "${var.env_prefix}-server"
87      }
88  }
89
90  output "ec2_public_ip" {
91      value = aws_instance.myapp-server.public_ip
92  }

```

By assigning the result of the -> sh "terraform output ec2_public_ip" to a env var that Jenkins can later access:

```

43  steps {
44      script {
45          dir('terraform') {
46              sh "terraform init"
47              sh "terraform apply -auto-approve"
48              EC2_PUBLIC_IP = sh "terraform output ec2_public_ip" You, 3 minutes ago
49          }
50      }
51  }
52  }

```

But the above doesn't really work, first we need to print out the sh result to the standard output in order to save it to a variable in this case an env var.

```

45         dir('terraform') {
46             sh "terraform init"
47             sh "terraform apply -auto-approve"
48             EC2_PUBLIC_IP = sh( You, A minute ago • Uncommitted changes
49                 script: "terraform output ec2_public_ip",
50                 returnStdout: true
51             ).trim()
52         }
53     }
54 }
55 }

```

Breaking it down:

- sh(...) — runs a shell command inside the pipeline
- script: "terraform output ec2_public_ip" — the command to run; asks Terraform for the value of the ec2_public_ip output variable
- returnStdout: true — instead of just running the command, capture what it prints to stdout and return it as a string. Without this, sh just returns the exit code (0 for success)
- .trim() — removes any trailing newline/whitespace that the shell command adds at the end
- EC2_PUBLIC_IP = — stores the result in a pipeline variable so you can use it later (e.g. to SSH into the instance)

So in plain English: run terraform output ec2_public_ip, grab what it prints, strip the whitespace, and store it in EC2_PUBLIC_IP.

Now we can use EC2_PUBLIC_IP in the 'deploy' stage


```

43     steps {
44         script {
45             dir('terraform') {
46                 sh "terraform init"
47                 sh "terraform apply -auto-approve"
48                 EC2_PUBLIC_IP = sh(
49                     script: "terraform output ec2_public_ip",
50                     returnStdout: true
51                 ).trim()
52             }
53         }
54     }
55 }
56 stage("deploy") {
57     steps {
58         script {
59             echo 'deploying docker image to EC2...'
60
61             def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
62             def ec2Instance = "ec2-user@${EC2_PUBLIC_IP}"
63
64             sshagent(['server-ssh-key']) {
65                 sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
66                 sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
67                 sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
68             }
69         }
70     }

```

From previous demos we noticed that after creating the EC2 instance using TF it took some time for the EC2 to be ready (up and running) it takes time to initialised.

We could see that in the AWS UI, where the 'instance state' was 'running' but the 'status check' was 'initializing'

Instances (1) Info							Refresh	Connect	Instance state	Actions	Launch
Find Instance by attribute or tag (case-sensitive)											
Instance state = running Clear filters											
☐	Name ✎	Instance ID	Instance state	Instance type	Status check	Alarm status					
☐	dev-server	i-0feeae90c1592af86	Running	t2.micro	Initializing	No alarms					

And once the initialisation is done then we see the green check marks displayed (2/2 check passed) like we see in this other example:

Instances (4) Info							Last updated 1 minute ago
Saved filter sets Choose filter set ▾		Find Instance by attribute or tag (case-sensitive)					
☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Put
☐	my-instance	i-0779fab597914e40	Terminated	t2.micro	-	eu-west-2b	-
☐		i-0c94659a8792f7a40	Terminated	t2.micro	-	eu-west-2c	-
☐	dev-server	i-0d95013d3bb086230	Terminated	t2.micro	-	eu-west-2a	-
☐	dev-server	i-02588624ff5fb6079	Running	t2.micro	2/2 checks passed	eu-west-2a	-

And part of the initialisation is to run the script

```

1 #!/bin/bash
2 sudo dnf update -y && sudo dnf install -y docker
3 sudo systemctl start docker
4 sudo usermod -aG docker ec2-user
5
6 #install docker-compose
7 sudo curl -SL "https://github.com/docker/compose/releases/download/v2.36.2/docker-compose-linux-x86_64" -o /usr/local/bin/docker-compose
8 sudo chmod +x /usr/local/bin/docker-compose

```

The thing is that as far as 'provision server' stage is concerned, TF has finished its work. AWS is taking time to finish the initialization. So jenkins file now moves to 'deploy' stage but in reality the server is not ready yet for deployment, it is still initialising.

```

46 sh "terraform init"
47 sh "terraform apply --auto-approve"
48 EC2_PUBLIC_IP = sh(
49     script: "terraform output ec2_public_ip",
50     returnStdout: true
51 ).trim()
52 }
53 }
54 }
55 }
56 stage("deploy") {
57     steps {
58         script {
59             echo 'deploying docker image to EC2...'
60
61             def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
62             def ec2Instance = "ec2-user@${EC2_PUBLIC_IP}"
63
64             sshagent(['ec2-server-key']) {
65                 sh "scp server-cmds.sh ${ec2Instance}:/home/ec2-user"
66                 sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} 'cd /home/ec2-user; ${shellCmd}'"
67             }
68         }
69     }
70 }
71 }
72 }
73 }

```

server created

entry-script.sh is executed

initializing...

deploy stage begins... ❌

still initializing...

So when jenkins tries to execute the commands of the 'deploy' stage it fails

```
53     }
54 }
55 }
56 stage("deploy") {
57     steps {
58         script {
59             echo 'deploying docker image to EC2...'
60
61             def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
62             def ec2Instance = "ec2-user@${EC2_PUBLIC_IP}"
63
64             sshagent(['ec2-server-key']) {
65                 sh "scp server-cmds.sh ${ec2Instance}:/home/ec2-user"
66                 sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
67             }
68         }
69     }
70 }
71 }
72 }
73 }
```

entry-script.sh is executed

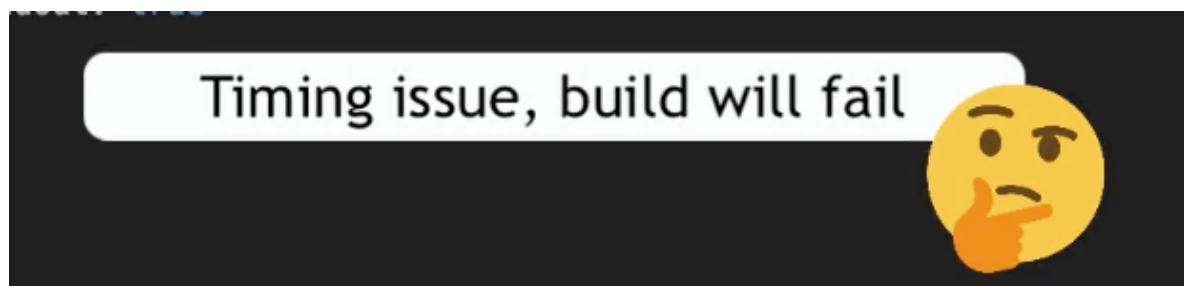
server created

initializing...

deploy stage begins... ❌

still initializing...

This is a **timing** issue 🕒



This will happen the first time Jenkins runs this pipeline as it will have to create the server but next time Jenkins runs the pipeline the server already exists so it won't be an issue.

Two options:

- Execute 'provision server' stage as the first stage
- Add a sleep to the 'deploy' stage before executing the commands to give some time for the initialisation of the server to be done.

So let's see the sleep code:

```

55     }
56     stage("deploy") {
57         steps {
58             script {
59                 sleep(time: 90, unit: "SECONDS")
60                 echo 'deploying docker image to EC2...'
61
62                 def shellCmd = "bash ./server-cmds.sh ${IMAGE_NAME}"
63                 def ec2Instance = "ec2-user@${EC2_PUBLIC_IP}"
64
65                 sshagent(['server-ssh-key']) {
66                     sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
67                     sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
68                     sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
69                 }
70             }

```

And we can add a couple of echo to log that we are waiting and the ec2 public ip see lines 59 and 63:

```

57         steps {
58             script {
59                 echo "waiting for EC2 server to initialize"
60                 sleep(time: 90, unit: "SECONDS")
61
62                 echo 'deploying docker image to EC2...'
63                 echo "${EC2_PUBLIC_IP}"

```

The above solves the timing issue but the code needs to be optimised. Once the server exists when we run the pipeline it will always wait for a minute and a half for no reason.

So to improve this we can wrap the sleep code in an IF statement that checks whether the Ec2 instance initialisation is done or not to decide whether to sleep or not.

Another adjustment is to add 'StrictHostKeyChecking=no' To all of the secure copies -> scp commands

To turn off the strict host key checking when we are connecting to the EC2 instance

```

67
68 sshagent(['server-ssh-key']) {
69   sh "scp -o server-cmds.sh ${ec2Instance}:/home/ec2-user"
70   sh "scp -o docker-compose.yaml ${ec2Instance}:/home/ec2-user"
71   sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
72 }
73 }
74

```

So now our file looks like this

```

67
68 sshagent(['server-ssh-key']) {
69   sh "scp -o StrictHostKeyChecking=no server-cmds.sh ${ec2Instance}:/home/ec2-user"
70   sh "scp -o StrictHostKeyChecking=no docker-compose.yaml ${ec2Instance}:/home/ec2-user"
71   sh "ssh -o StrictHostKeyChecking=no ${ec2Instance} ${shellCmd}"
72 }

```

We created sshagent and the 'ec2-server-key' in Module 9 - chapter 8

Top Hits

Module 12 - chapter 23
15:25 ...we need via **sshagent** (see lines 49-52) So w...
DevOps_Nano — ICloud

Module 9 - chapter 8
15:30 ...block inside **sshagent** in the jenkinsfile will ne...
DevOps_Nano — ICloud

Notes

Module 12 - chapter 15
12:19 Provisioners in Terraform The project files used l...
DevOps_Nano — ICloud

Module 12 - chapter 22
Yesterday ...jenkinsfile-**sshagent** I added the starting-...
DevOps_Nano — ICloud

Module 9 - chapter 9
Yesterday ...in the **sshagent** block -> sh "scp docker-...
DevOps_Nano — ICloud

Module 3 - chapter 3
21/09/2025 Found in attachments
DevOps_Nano — ICloud

Attachments

Adding your SSH key to the ssh-agent
PNG image - 134 KB

System	global	global	global
System	global	github-token	github-token
System	global	github-credentials	AdminCatalyst

Stores scoped to aws-multibranch-pipeline

P	Store	Domains
aws-multibranch-pipeline	global	

Stores from parent

P	Store	Domains
System	global	

Icon: S M L

Now we create the SSH credential:

Jenkins aws-multibranch-pipeline / Credentials / Folder / Global credentials (unrestrict...

New credentials

Kind: SSH Username with private key

ID: ec2-server-key

Description:

Username: ec2-user

Private Key: ☐ Enter directly ☒ Key

'server-ssh-key' is the one we created at the beginning in my 'terraform-pipeline'

Jenkins terraform-pipeline / Credentials / Folder / Global credentials (unrestrict...

Global credentials (unrestricted) + Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
server-ssh-key	ec2-user	SSH Username with private key	

Icon: S M L